

✓ Import Libraries

We import necessary libraries for data manipulation, preprocessing, modeling, evaluation, and explainability, including pandas, scikit-learn, matplotlib, seaborn, and SHAP.

```
!pip install catboost
```

```
Collecting catboost
  Using cached catboost-1.2.8-cp311-cp311-manylinux2014_x86_64.whl.metadata (1.2 kB)
Requirement already satisfied: graphviz in /usr/local/lib/python3.11/dist-packages (from catboost) (0.20.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (from catboost) (3.10.0)
Requirement already satisfied: numpy<3.0, >=1.16.0 in /usr/local/lib/python3.11/dist-packages (from catboost) (2.0.2)
Requirement already satisfied: pandas>=0.24 in /usr/local/lib/python3.11/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from catboost) (1.15.2)
Requirement already satisfied: plotly in /usr/local/lib/python3.11/dist-packages (from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.11/dist-packages (from catboost) (1.17.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2025.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (1.3.2)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (4.57.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (24.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (11.2.1)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (3.2.3)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly->catboost) (9.1.2)
Using cached catboost-1.2.8-cp311-cp311-manylinux2014_x86_64.whl (99.2 MB)
Installing collected packages: catboost
Successfully installed catboost-1.2.8
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from xgboost import XGBClassifier
from sklearn.svm import SVC
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.ensemble import AdaBoostClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_matrix, classification_report,
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
```

✓ Load Dataset

We load the customer dataset containing financial, demographic, and behavioral features, which we will use to classify credit scores into Bad, Standard, or Good categories.

```
df = pd.read_csv('/content/Bank Data.csv')
```

```
df.head()
```

02/05/2025, 22:22

Classifying The Credit Scores.ipynb - Colab

	ID	Customer_ID	Month	Name	Age	SSN	Occupation	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	...	Num_Cr
0	0x160a	CUS_0xd40	September	Aaron Maashoh	23	821-00-0265	Scientist	19114.12	1824.843333	3.0	...	
1	0x160b	CUS_0xd40	October	Aaron Maashoh	24	821-00-0265	Scientist	19114.12	1824.843333	3.0	...	
2	0x160c	CUS_0xd40	November	Aaron Maashoh	24	821-00-0265	Scientist	19114.12	1824.843333	3.0	...	
3	0x160d	CUS_0xd40	December	Aaron Maashoh	24	821-00-0265	Scientist	19114.12	NaN	3.0	...	
4	0x1616	CUS_0x21b1	September	Rick Rothackerj	28	004-07-5839		34847.84	3037.986667	2.0	...	

5 rows × 27 columns

Data Preprocessing

We apply preprocessing steps:

- Drop non-informative columns (e.g., ID, Customer_ID, Name, SSN)
- Handle missing values:
 - Fill numerical with median
 - Fill categorical with mode or "Unknown"
- Encode categorical variables:
 - Label encode ordinal features (Payment_Behaviour, Credit_Mix, Occupation)
 - One-hot encode multi-label features (Type_of_Loan)
- Cap outliers using the IQR method
- Encode target: Bad = 0, Standard = 1, Good = 2

Exploratory Data Analysis (EDA)

We explore the dataset to understand distributions and relationships:

- Check class distribution (Credit_Mix: Standard, Good, Bad)
- Visualize important features with count plots, box plots, and heatmaps
- Identify top features using feature importance and SHAP

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13685 entries, 0 to 13684
Data columns (total 27 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   ID                                    13685 non-null  object
1   Customer_ID                          13685 non-null  object
2   Month                                13684 non-null  object
3   Name                                  12257 non-null  object
4   Age                                   13684 non-null  object
5   SSN                                   13684 non-null  object
6   Occupation                            13684 non-null  object
7   Annual_Income                         13684 non-null  object
8   Monthly_Inhand_Salary                 11622 non-null  float64
9   Num_Bank_Accounts                     13684 non-null  float64
10  Num_Credit_Card                       13684 non-null  float64
11  Interest_Rate                         13684 non-null  float64
12  Num_of_Loan                           13684 non-null  object
13  Type_of_Loan                           12096 non-null  object
14  Delay_from_due_date                   13684 non-null  float64
15  Num_of_Delayed_Payment                12709 non-null  object
16  Changed_Credit_Limit                  13684 non-null  object
17  Num_Credit_Inquiries                  13398 non-null  float64
18  Credit_Mix                            13684 non-null  object
```

```
19 Outstanding_Debt      13684 non-null object
20 Credit_Utilization_Ratio 13684 non-null float64
21 Credit_History_Age      12440 non-null object
22 Payment_of_Min_Amount   13684 non-null object
23 Total_EMI_per_month     13684 non-null float64
24 Amount_invested_monthly 13087 non-null object
25 Payment_Behaviour       13684 non-null object
26 Monthly_Balance         13521 non-null object
dtypes: float64(8), object(19)
memory usage: 2.8+ MB
```

```
df.describe()
```



	Monthly_Inhand_Salary	Num_Bank_Accounts	Num_Credit_Card	Interest_Rate	Delay_from_due_date	Num_Credit_Inquiries	Credit_Utili
count	11622.000000	13684.000000	13684.000000	13684.000000	13684.000000	13398.000000	
mean	4268.421084	16.162672	23.713315	65.859836	20.944315	32.675698	
std	3226.322375	111.236393	131.924455	437.410829	14.619822	208.072727	
min	333.596667	0.000000	0.000000	1.000000	-5.000000	0.000000	
25%	1650.781667	3.000000	4.000000	7.000000	10.000000	4.000000	
50%	3143.796667	6.000000	6.000000	13.000000	18.000000	7.000000	
75%	6098.132083	7.000000	7.000000	20.000000	28.000000	10.000000	
max	14866.446667	1769.000000	1499.000000	5799.000000	67.000000	2593.000000	



```
df.shape
```



```
(13685, 27)
```

```
df.duplicated().sum()
```



```
np.int64(0)
```

```
df.isnull().sum()
```



	0
ID	0
Customer_ID	0
Month	1
Name	1428
Age	1
SSN	1
Occupation	1
Annual_Income	1
Monthly_Inhand_Salary	2063
Num_Bank_Accounts	1
Num_Credit_Card	1
Interest_Rate	1
Num_of_Loan	1
Type_of_Loan	1589
Delay_from_due_date	1
Num_of_Delayed_Payment	976
Changed_Credit_Limit	1
Num_Credit_Inquiries	287
Credit_Mix	1
Outstanding_Debt	1
Credit_Utilization_Ratio	1
Credit_History_Age	1245
Payment_of_Min_Amount	1
Total_EMI_per_month	1
Amount_invested_monthly	598
Payment_Behaviour	1
Monthly_Balance	164



```
df['Credit_Mix'].unique()  
df['Credit_Mix'].value_counts()
```



	count
Credit_Mix	
Standard	4931
Good	3391
_	2780
Bad	2582



```
df['Credit_Mix'].unique()  
  
 array(['Good', '_', 'Standard', 'Bad', nan], dtype=object)  
  
df['Payment_Behaviour'].unique()  
df['Payment_of_Min_Amount'].unique()
```

```
 array(['No', 'Yes', 'NM', nan], dtype=object)
```

```
df['Payment_Behaviour'].unique()
```

```
array(['Low_spent_Small_value_payments',
      'High_spent_Medium_value_payments',
      'Low_spent_Medium_value_payments',
      'High_spent_Large_value_payments',
      'Low_spent_Large_value_payments', '!@9#%8',
      'High_spent_Small_value_payments', nan], dtype=object)
```

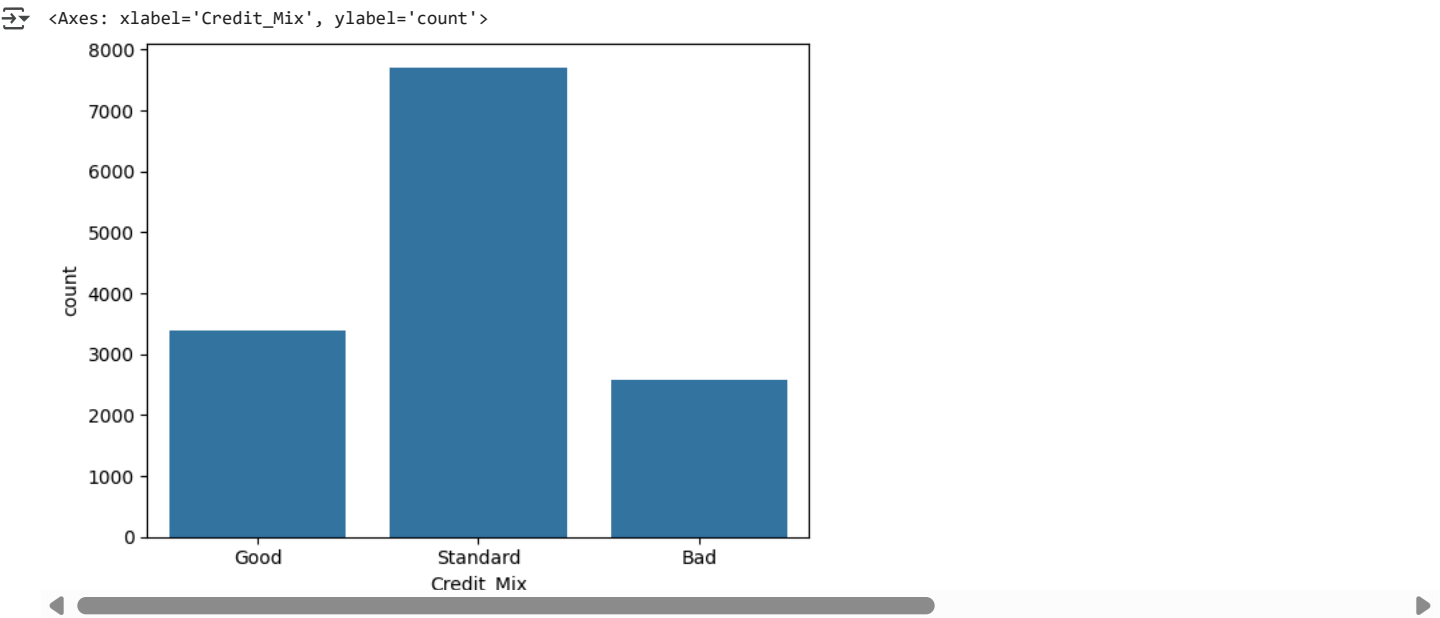
```
columns_to_drop = ['ID', 'Customer_ID', 'Name', 'SSN', 'Month']
df.drop(columns=columns_to_drop, inplace=True)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13685 entries, 0 to 13684
Data columns (total 22 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Age                                   13684 non-null  object
1   Occupation                           13684 non-null  object
2   Annual_Income                         13684 non-null  object
3   Monthly_Inhand_Salary                11622 non-null  float64
4   Num_Bank_Accounts                    13684 non-null  float64
5   Num_Credit_Card                       13684 non-null  float64
6   Interest_Rate                        13684 non-null  float64
7   Num_of_Loan                          13684 non-null  object
8   Type_of_Loan                         12096 non-null  object
9   Delay_from_due_date                  13684 non-null  float64
10  Num_of_Delayed_Payment                12709 non-null  object
11  Changed_Credit_Limit                  13684 non-null  object
12  Num_Credit_Inquiries                  13398 non-null  float64
13  Credit_Mix                           13684 non-null  object
14  Outstanding_Debt                     13684 non-null  object
15  Credit_Utilization_Ratio              13684 non-null  float64
16  Credit_History_Age                    12440 non-null  object
17  Payment_of_Min_Amount                 13684 non-null  object
18  Total_EMI_per_month                   13684 non-null  float64
19  Amount_invested_monthly               13087 non-null  object
20  Payment_Behaviour                     13684 non-null  object
21  Monthly_Balance                       13521 non-null  object
dtypes: float64(8), object(14)
memory usage: 2.3+ MB
```

```
mode_value = df.loc[df['Credit_Mix'] != '_', 'Credit_Mix'].mode()[0]
df['Credit_Mix'] = df['Credit_Mix'].replace('_', mode_value)
```

```
df['Credit_Mix'].value_counts()
sns.countplot(x='Credit_Mix', data=df)
```



df.head()

	Age	Occupation	Annual_Income	Monthly_Inhand_Salary	Num_Bank_Accounts	Num_Credit_Card	Interest_Rate	Num_of_Loan	Type_of_Loan	
0	23	Scientist	19114.12	1824.843333	3.0	4.0	3.0	4	Auto Loan, Credit-Builder Loan, Personal Loan,...	
1	24	Scientist	19114.12	1824.843333	3.0	4.0	3.0	4	Auto Loan, Credit-Builder Loan, Personal Loan,...	
2	24	Scientist	19114.12	1824.843333	3.0	4.0	3.0	4	Auto Loan, Credit-Builder Loan, Personal Loan,...	
3	24_	Scientist	19114.12	NaN	3.0	4.0	3.0	4	Auto Loan, Credit-Builder Loan, Personal Loan,...	
4	28		34847.84	3037.986667	2.0	4.0	6.0	1	Credit-Builder Loan	

5 rows × 22 columns

```
def clean_numeric(col):
    return pd.to_numeric(df[col].astype(str).str.replace(',', '').str.strip(), errors='coerce')

numeric_columns = ['Age', 'Annual_Income', 'Num_of_Loan', 'Num_of_Delayed_Payment',
                    'Changed_Credit_Limit', 'Outstanding_Debt', 'Amount_invested_monthly', 'Monthly_Balance']

for col in numeric_columns:
    df[col] = clean_numeric(col)

df.info()

df['Payment_of_Min_Amount'].unique()

array(['No', 'Yes', 'NM', nan], dtype=object)
```

```
df['Payment_Behaviour'].unique()
```

```
array(['Low_spent_Small_value_payments',
      'High_spent_Medium_value_payments',
      'Low_spent_Medium_value_payments',
      'High_spent_Large_value_payments',
      'Low_spent_Large_value_payments', '!@9#%8',
      'High_spent_Small_value_payments', nan], dtype=object)
```

```
df['Type_of_Loan'].unique()
```

```
array(['Auto Loan, Credit-Builder Loan, Personal Loan, and Home Equity Loan',
      'Credit-Builder Loan', 'Auto Loan, Auto Loan, and Not Specified',
      ...,
      'Debt Consolidation Loan, Payday Loan, Auto Loan, Mortgage Loan, and Not Specified',
      'Credit-Builder Loan, Personal Loan, Student Loan, and Credit-Builder Loan',
      'Payday Loan, Home Equity Loan, Auto Loan, Mortgage Loan, and Credit-Builder Loan'],
      dtype=object)
```

```
df['Occupation'].unique()
```

```
array(['Scientist', '_____', 'Teacher', 'Engineer', 'Entrepreneur',
      'Developer', 'Lawyer', 'Media_Manager', 'Doctor', 'Journalist',
      'Manager', 'Accountant', 'Musician', 'Mechanic', 'Writer',
      'Architect', nan], dtype=object)
```

```
df.isnull().sum()
```

Show hidden output

```
num_cols = ['Age', 'Annual_Income', 'Num_of_Loan', 'Num_of_Delayed_Payment',
            'Changed_Credit_Limit', 'Num_Credit_Inquiries', 'Outstanding_Debt',
            'Amount_invested_monthly', 'Monthly_Balance']
```

```
for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)
```

<ipython-input-26-a1c4464ff7eb>:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy. For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)'

```
df[col].fillna(df[col].median(), inplace=True)
```

```
df[['Annual_Income', 'Monthly_Inhand_Salary']].corr()
```

	Annual_Income	Monthly_Inhand_Salary
Annual_Income	1.000000	0.028903
Monthly_Inhand_Salary	0.028903	1.000000

```
median_salary = df['Monthly_Inhand_Salary'].median()
df['Monthly_Inhand_Salary'].fillna(median_salary, inplace=True)
```

<ipython-input-28-8a64ee51e364>:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy. For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)'

```
df['Monthly_Inhand_Salary'].fillna(median_salary, inplace=True)
```

```
import numpy as np
```

```
def convert_credit_history(x):
    try:
        years, months = 0, 0
        if 'Years' in x:
```

```

        years = int(x.split('Years')[0].strip())
        if 'Months' in x:
            months = int(x.split('Years')[1].split('Months')[0].strip())
        elif 'Months' in x:
            months = int(x.split('Months')[0].strip())
        return years * 12 + months
    except:
        return np.nan

```

```
df['Credit_History_Age'] = df['Credit_History_Age'].astype(str).apply(convert_credit_history)
```

```

median_value = df['Credit_History_Age'].median()
df['Credit_History_Age'].fillna(median_value, inplace=True)

```

 <ipython-input-29-4d1e3f319aa3>:22: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
df['Credit_History_Age'].fillna(median_value, inplace=True)
```

```
df['Occupation'].value_counts()
```

 [Show hidden output](#)

```

df['Occupation'] = df['Occupation'].replace('_____', 'Unknown')
df['Occupation_missing'] = df['Occupation'].apply(lambda x: 1 if x == 'Unknown' else 0)
from sklearn.preprocessing import LabelEncoder

```

```

le = LabelEncoder()
df['Occupation'] = le.fit_transform(df['Occupation'])

print(dict(zip(le.classes_, le.transform(le.classes_))))

```

 {'Accountant': np.int64(0), 'Architect': np.int64(1), 'Developer': np.int64(2), 'Doctor': np.int64(3), 'Engineer': np.int64(4), 'Entrepreneur': np.int64(5), 'Lawyer': np.int64(6), 'Manager': np.int64(7), 'Marketing': np.int64(8), 'Nurse': np.int64(9), 'Other': np.int64(10), 'Sales': np.int64(11), 'Self-employed': np.int64(12), 'Student': np.int64(13), 'Teacher': np.int64(14), 'Unemployed': np.int64(15), 'Unknown': np.int64(16), 'Writer': np.int64(17)}

```

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['Payment_of_Min_Amount'] = le.fit_transform(df['Payment_of_Min_Amount'])

```

```

mode_pb = df['Payment_Behaviour'].mode()[0]
df['Payment_Behaviour'].replace('@##%', mode_pb, inplace=True)

```

```

mapping = {
    'Low_spent_Small_value_payments': 0,
    'Low_spent_Medium_value_payments': 1,
    'Low_spent_Large_value_payments': 2,
    'High_spent_Small_value_payments': 3,
    'High_spent_Medium_value_payments': 4,
    'High_spent_Large_value_payments': 5
}

```

```
df['Payment_Behaviour'] = df['Payment_Behaviour'].map(mapping)
```

```

mode_value = df['Payment_Behaviour'].mode()[0]
df['Payment_Behaviour'].fillna(mode_value, inplace=True)

```

 <ipython-input-33-38990cd0d8d8>:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```

df['Payment_Behaviour'].replace('@##%', mode_pb, inplace=True)
<ipython-input-33-38990cd0d8d8>:16: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.

```


The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting value

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].me

```
df['Payment_Behaviour'].fillna(mode_value, inplace=True)
```

```
all_loans = set()
for row in df['Type_of_Loan'].dropna():
    loans = [loan.strip() for loan in row.split(',')]
    all_loans.update(loans)

loan_types_df = pd.DataFrame(sorted(all_loans), columns=['Loan_Type'])

print(loan_types_df)
```

```

Loan_Type
0      Auto Loan
1  Credit-Builder Loan
2  Debt Consolidation Loan
3    Home Equity Loan
4    Mortgage Loan
5    Not Specified
6    Payday Loan
7    Personal Loan
8    Student Loan
9    and Auto Loan
10  and Credit-Builder Loan
11  and Debt Consolidation Loan
12  and Home Equity Loan
13  and Mortgage Loan
14  and Not Specified
15  and Payday Loan
16  and Personal Loan
17  and Student Loan
```

```
df['Type_of_Loan'] = df['Type_of_Loan'].str.replace(', and', ', ', regex=False)
```

```
df['Type_of_Loan'] = df['Type_of_Loan'].str.replace('and ', ', ', regex=False)
```

```
all_loans = set()
for row in df['Type_of_Loan'].dropna():
    loans = [loan.strip() for loan in row.split(',')]
    all_loans.update(loans)

loan_types_df = pd.DataFrame(sorted(all_loans), columns=['Loan_Type'])

print(loan_types_df)
```

```

Loan_Type
0      Auto Loan
1  Credit-Builder Loan
2  Debt Consolidation Loan
3    Home Equity Loan
4    Mortgage Loan
5    Not Specified
6    Payday Loan
7    Personal Loan
8    Student Loan
```

```
df['Type_of_Loan'] = df['Type_of_Loan'].fillna('Not Specified')
df['Type_of_Loan'] = df['Type_of_Loan'].apply(
    lambda x: ', '.join(sorted(set([i.strip() for i in x.split(',')]))))
)
```

```
all_loans = set()
for row in df['Type_of_Loan']:
    loans = [loan.strip() for loan in row.split(',')]
    all_loans.update(loans)
all_loans = sorted(all_loans)
print(all_loans)
```

↻ ['Auto Loan', 'Credit-Builder Loan', 'Debt Consolidation Loan', 'Home Equity Loan', 'Mortgage Loan', 'Not Specified', 'Payday Loan', 'Pe

```
for loan in all_loans:
    df[loan] = df['Type_of_Loan'].apply(lambda x: 1 if loan in x else 0)
df.drop(columns=['Type_of_Loan'], inplace=True)
```

```
df.info()
```

↻ Show hidden output

```
df.isnull().sum()
```

↻ Show hidden output

```
mapping = {'Bad': 0, 'Standard': 1, 'Good': 2}
df['Credit_Mix'] = df['Credit_Mix'].map(mapping)
```

```
print(df['Credit_Mix'].unique())
```

↻ [2. 1. 0. nan]

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

```
for col in df.select_dtypes(include=['float64', 'int64']).columns:
    print(f"\n--- Checking column: {col} ---")

    print(df[col].describe())

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
    num_outliers = outliers.shape[0]
    print(f"Number of outliers in '{col}': {num_outliers}")

    plt.figure(figsize=(8, 4))
    sns.boxplot(x=df[col])
    plt.title(f'Boxplot of {col}')
    plt.show()
```

↻ Show hidden output

```
min_values = df.min()
max_values = df.max()

summary = pd.DataFrame({'Min': min_values, 'Max': max_values})
print(summary)
```

↻ Show hidden output

```
import numpy as np
```

```
continuous_cols = [
    'Age', 'Annual_Income', 'Monthly_Inhand_Salary',
    'Num_Bank_Accounts', 'Num_Credit_Card', 'Interest_Rate',
    'Num_of_Loan', 'Delay_from_due_date', 'Num_of_Delayed_Payment',
    'Changed_Credit_Limit', 'Num_Credit_Inquiries', 'Outstanding_Debt',
    'Credit_Utilization_Ratio', 'Total_EMI_per_month',
    'Amount_invested_monthly', 'Monthly_Balance'
```

```

]

for col in continuous_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    df[col] = np.where(df[col] < lower_bound, lower_bound, df[col])
    df[col] = np.where(df[col] > upper_bound, upper_bound, df[col])

    print(f"Column: {col} → Applied capping at {lower_bound:.2f} and {upper_bound:.2f}")

print("\n Outlier capping complete. Dataset shape remains:", df.shape)

```

```

↗ Column: Age → Applied capping at -0.50 and 67.50
Column: Annual_Income → Applied capping at -56848.30 and 148709.30
Column: Monthly_Inhand_Salary → Applied capping at -3772.75 and 11154.16
Column: Num_Bank_Accounts → Applied capping at -3.00 and 13.00
Column: Num_Credit_Card → Applied capping at -0.50 and 11.50
Column: Interest_Rate → Applied capping at -12.50 and 39.50
Column: Num_of_Loan → Applied capping at -2.50 and 9.50
Column: Delay_from_due_date → Applied capping at -17.00 and 55.00
Column: Num_of_Delayed_Payment → Applied capping at -4.50 and 31.50
Column: Changed_Credit_Limit → Applied capping at -8.16 and 27.76
Column: Num_Credit_Inquiries → Applied capping at -5.00 and 19.00
Column: Outstanding_Debt → Applied capping at -1395.67 and 3773.37
Column: Credit_Utilization_Ratio → Applied capping at 15.52 and 48.93
Column: Total_EMI_per_month → Applied capping at -189.47 and 400.53
Column: Amount_invested_monthly → Applied capping at -142.41 and 442.72
Column: Monthly_Balance → Applied capping at -21.50 and 763.18

```

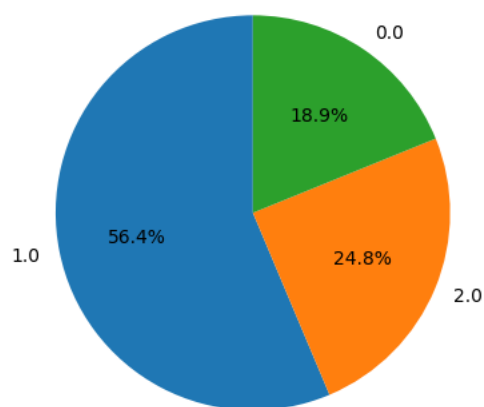
Outlier capping complete. Dataset shape remains: (13685, 31)

```

df['Credit_Mix'].value_counts().plot.pie(autopct='%1.1f%%', startangle=90)
plt.title('Credit_Mix Distribution')
plt.ylabel('')
plt.show()

```

↗ Credit_Mix Distribution



```

X = df.drop('Credit_Mix', axis=1)
y = df['Credit_Mix']

```

```

# Check for NaNs in y
print(y.isnull().sum())

```

↗ 1

```

y = y.fillna(y.mode()[0])

```

✓ Train-Test Split

We split the dataset into training and test sets using stratified sampling to maintain the class distribution across splits.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

✓ Model Building

We train multiple classification models to predict credit score:

- Logistic Regression
- Random Forest
- XGBoost
- Gradient Boosting
- AdaBoost
- LightGBM
- CatBoost

Model Evaluation

We evaluate models using accuracy, precision, recall, and F1-score, and compare their performance to identify the best-performing approach.

```
from sklearn.preprocessing import label_binarize
from sklearn.metrics import (
    accuracy_score, classification_report, confusion_matrix, roc_curve, auc
)
from sklearn.impute import SimpleImputer
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

# ✓ Create model dictionary
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000),
    'Random Forest': RandomForestClassifier(),
    'XGBoost': XGBClassifier(use_label_encoder=False, eval_metric='mlogloss'),
    'Gradient Boosting': GradientBoostingClassifier(),
    'AdaBoost': AdaBoostClassifier(),
    'LightGBM': LGBMClassifier(),
    'CatBoost': CatBoostClassifier(verbose=0)
}

# ✓ Handle missing values in X_train, X_test
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='median')
X_train_imputed = imputer.fit_transform(X_train)
X_test_imputed = imputer.transform(X_test)

# ✓ Binarize y_test for multiclass ROC
classes = sorted(y.unique())
y_test_bin = label_binarize(y_test, classes=classes)

# ✓ Loop over models
for name, model in models.items():
    print(f"\nModel: {name}")

    # Train
    model.fit(X_train_imputed, y_train)

    # Predict
    y_pred = model.predict(X_test_imputed)
```

```
y_proba = model.predict_proba(X_test_imputed)

# Metrics
acc = accuracy_score(y_test, y_pred)
print("Accuracy:", acc)
print("Classification Report:\n", classification_report(y_test, y_pred))
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

#  Confusion Matrix plot
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title(f'Confusion Matrix: {name}')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()

#  ROC curves and AUC
fpr, tpr, roc_auc = dict(), dict(), dict()
for i, cls in enumerate(classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_proba[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

plt.figure()
for i, cls in enumerate(classes):
    plt.plot(fpr[i], tpr[i], label=f'Class {cls} (AUC = {roc_auc[i]:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.title(f'ROC Curve: {name}')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```



Model: Logistic Regression

/usr/local/lib/python3.11/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

n_iter_i = _check_optimize_result(

Accuracy: 0.7135549872122762

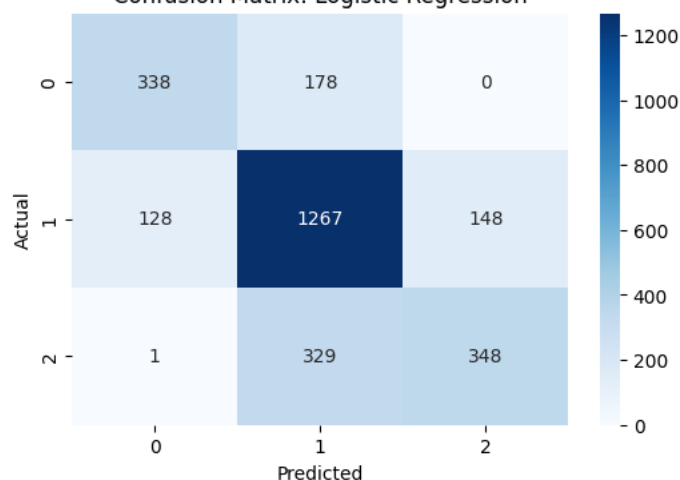
Classification Report:

	precision	recall	f1-score	support
0.0	0.72	0.66	0.69	516
1.0	0.71	0.82	0.76	1543
2.0	0.70	0.51	0.59	678
accuracy			0.71	2737
macro avg	0.71	0.66	0.68	2737
weighted avg	0.71	0.71	0.71	2737

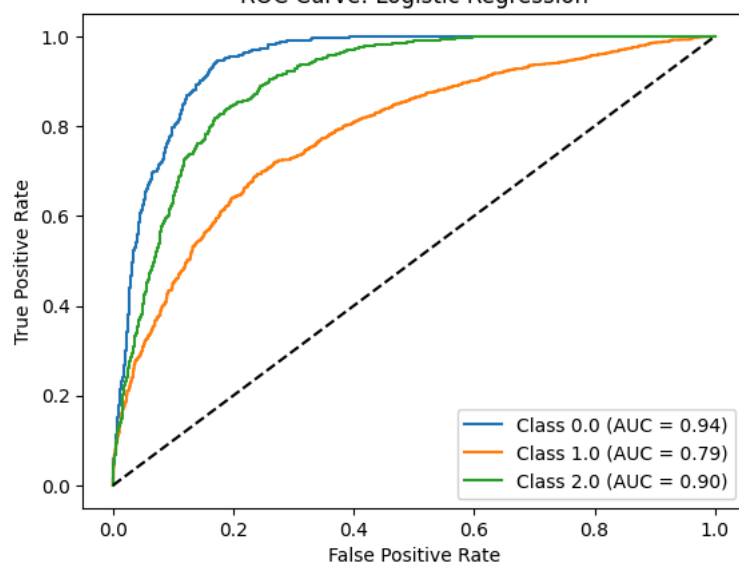
Confusion Matrix:

```
[[ 338  178   0]
 [ 128 1267  148]
 [   1  329  348]]
```

Confusion Matrix: Logistic Regression



ROC Curve: Logistic Regression



Model: Random Forest

Accuracy: 0.8545853123858239

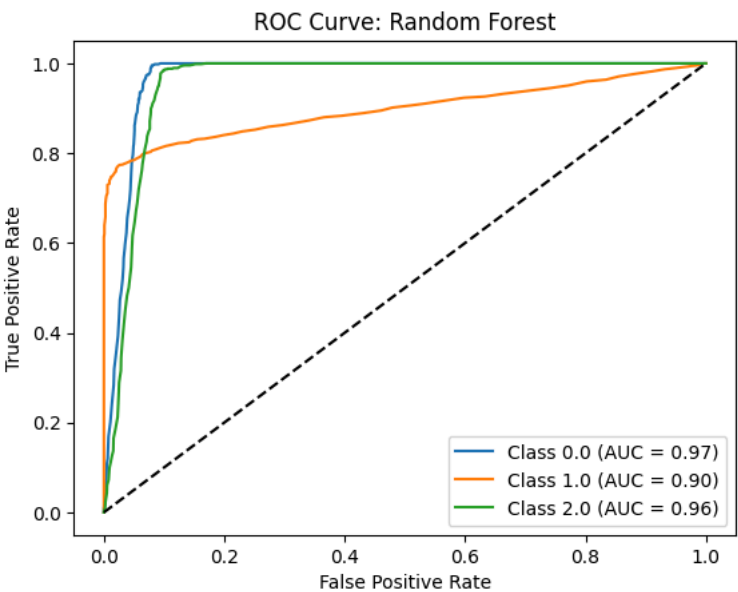
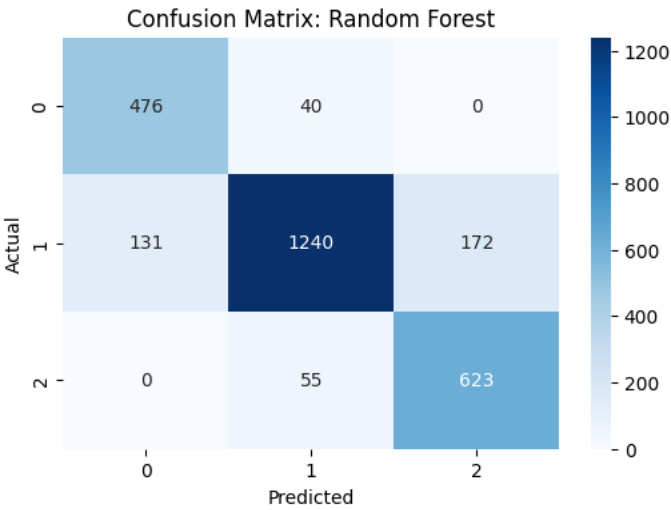
Classification Report:

	precision	recall	f1-score	support
0.0	0.78	0.92	0.85	516
1.0	0.93	0.80	0.86	1543
2.0	0.78	0.92	0.85	678

accuracy			0.85	2737
macro avg	0.83	0.88	0.85	2737
weighted avg	0.87	0.85	0.86	2737

Confusion Matrix:

[	476	40	0]
[	131	1240	172]
[	0	55	623]



Model: XGBoost
/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [16:19:05] WARNING: /workspace/src/learner.cc:740:
Parameters: { "use_label_encoder" } are not used.

warnings.warn(msg, UserWarning)

Accuracy: 0.8381439532334674

Classification Report:

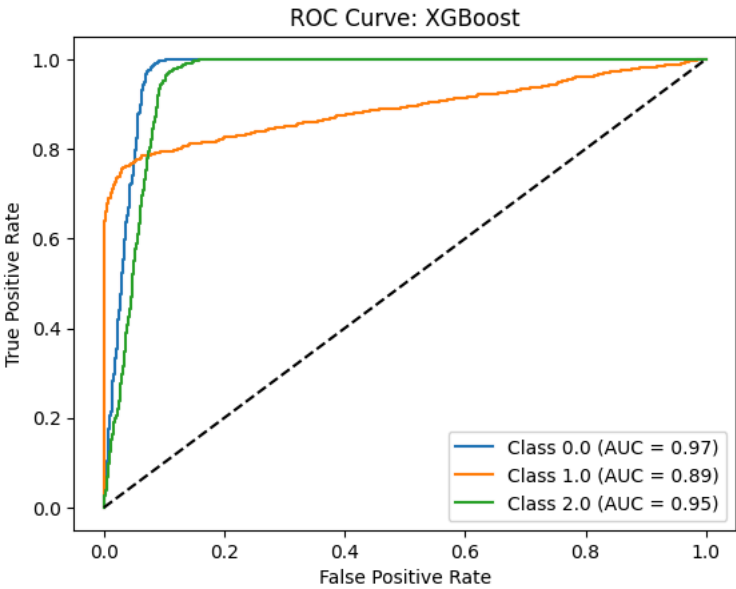
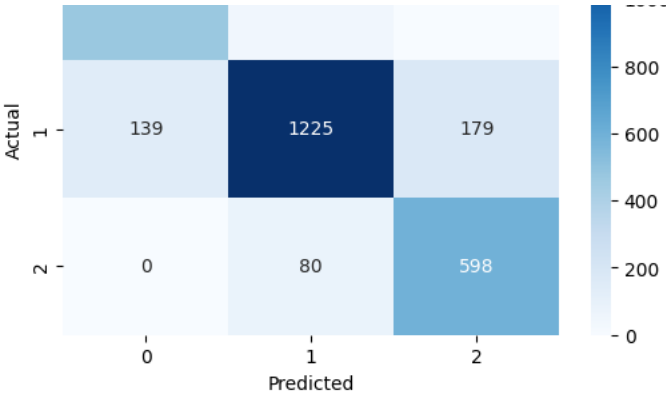
	precision	recall	f1-score	support
0.0	0.77	0.91	0.84	516
1.0	0.91	0.79	0.85	1543
2.0	0.77	0.88	0.82	678

accuracy			0.84	2737
macro avg	0.82	0.86	0.84	2737
weighted avg	0.85	0.84	0.84	2737

Confusion Matrix:

[	471	45	0]
[	139	1225	179]
[	0	80	598]

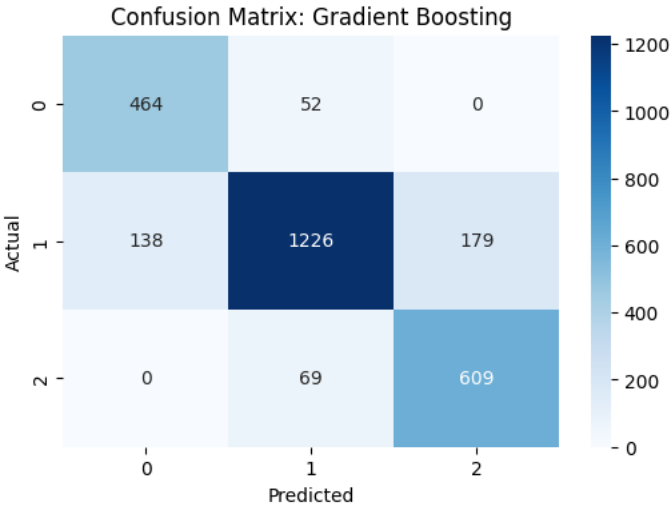


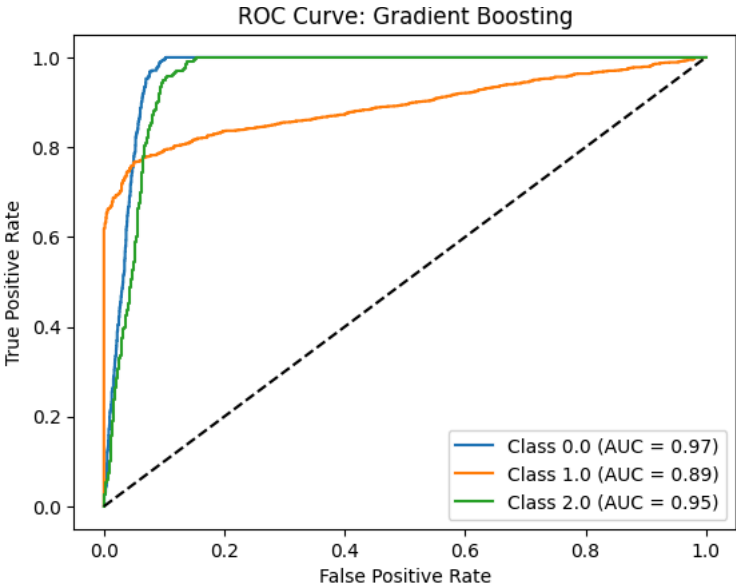


Model: Gradient Boosting
Accuracy: 0.8399707709170625
Classification Report:

	precision	recall	f1-score	support
0.0	0.77	0.90	0.83	516
1.0	0.91	0.79	0.85	1543
2.0	0.77	0.90	0.83	678
accuracy			0.84	2737
macro avg	0.82	0.86	0.84	2737
weighted avg	0.85	0.84	0.84	2737

Confusion Matrix:
[[464 52 0]
 [138 1226 179]
 [0 69 609]]



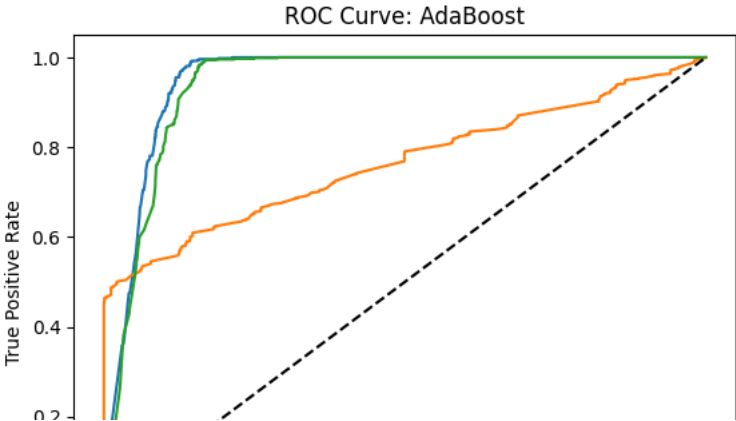
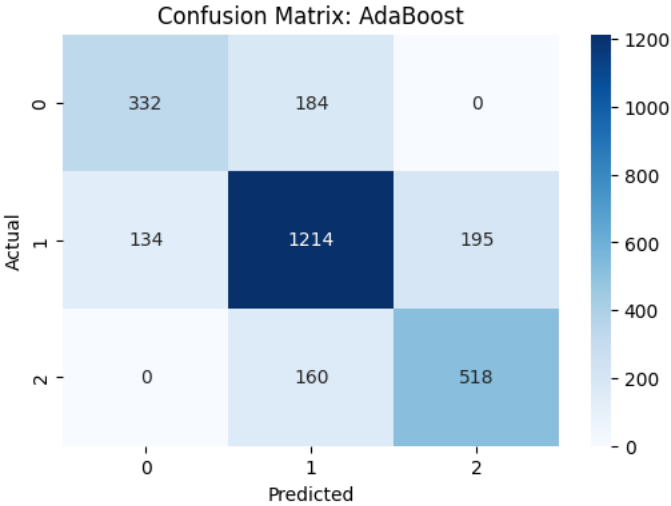


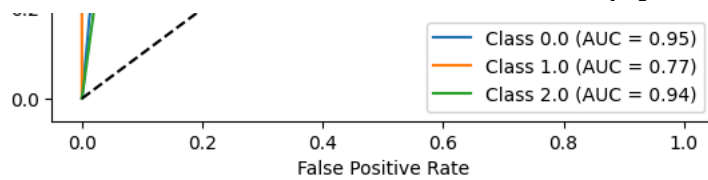
Model: AdaBoost
Accuracy: 0.7541103397880892
Classification Report:

	precision	recall	f1-score	support
0.0	0.71	0.64	0.68	516
1.0	0.78	0.79	0.78	1543
2.0	0.73	0.76	0.74	678
accuracy			0.75	2737
macro avg	0.74	0.73	0.73	2737
weighted avg	0.75	0.75	0.75	2737

Confusion Matrix:

```
[[ 332  184    0]
 [ 134 1214  195]
 [   0  160  518]]
```





Model: LightGBM

[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.003063 seconds.

You can set `force\_row\_wise=true` to remove the overhead.

And if memory is not enough, you can set `force\_col\_wise=true`.

[LightGBM] [Info] Total Bins 2313

[LightGBM] [Info] Number of data points in the train set: 10948, number of used features: 29

[LightGBM] [Info] Start training from score -1.667542

[LightGBM] [Info] Start training from score -0.573620

[LightGBM] [Info] Start training from score -1.395102

/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' warnings.warn(

/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' warnings.warn(

/usr/local/lib/python3.11/dist-packages/sklearn/utils/deprecation.py:151: FutureWarning: 'force_all_finite' was renamed to 'ensure_all_finite' warnings.warn(

Accuracy: 0.8410668615272195

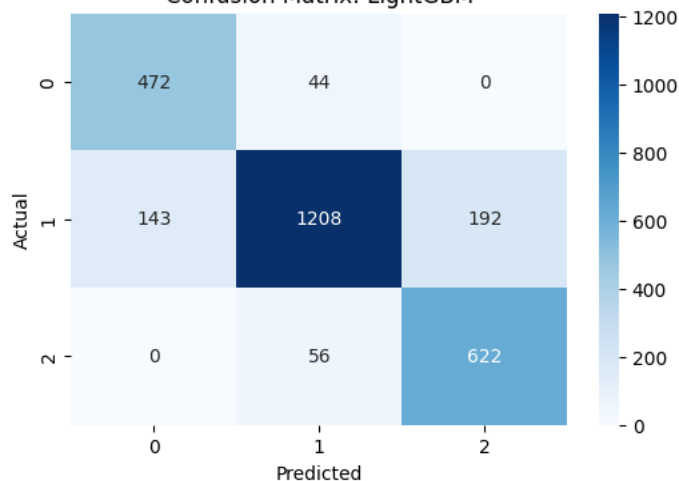
Classification Report:

	precision	recall	f1-score	support
0.0	0.77	0.91	0.83	516
1.0	0.92	0.78	0.85	1543
2.0	0.76	0.92	0.83	678
accuracy			0.84	2737
macro avg	0.82	0.87	0.84	2737
weighted avg	0.85	0.84	0.84	2737

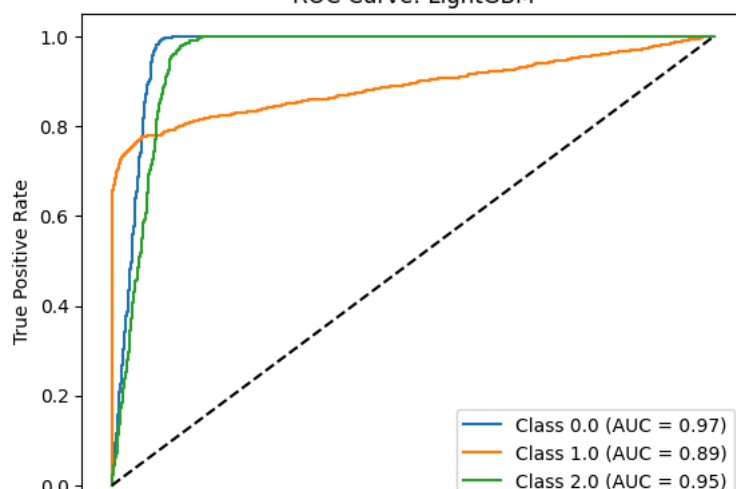
Confusion Matrix:

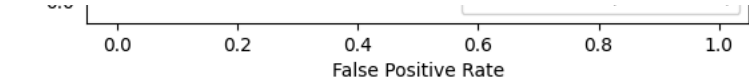
```
[[ 472  44   0]
 [ 143 1208 192]
 [   0   56 622]]
```

Confusion Matrix: LightGBM



ROC Curve: LightGBM

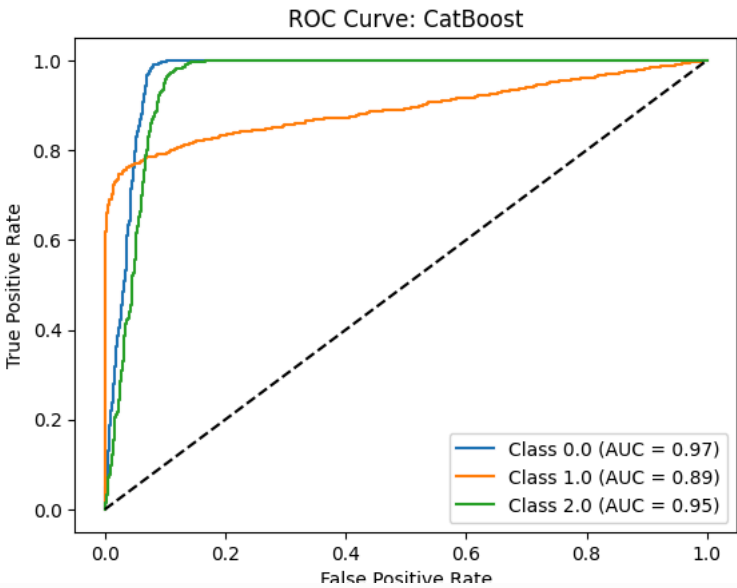
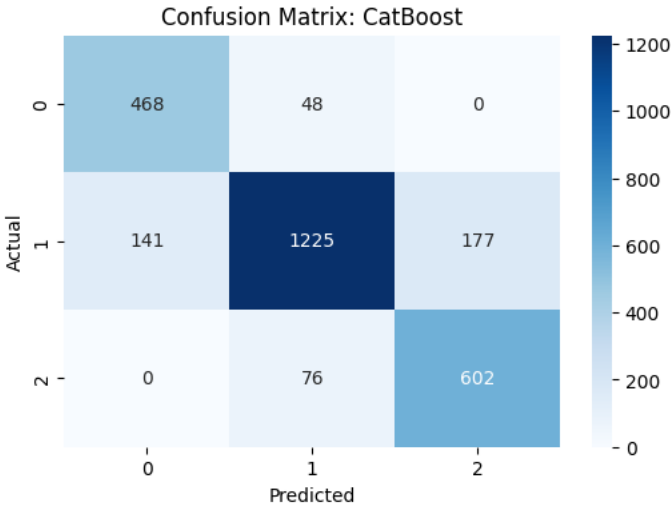




Model: CatBoost
Accuracy: 0.8385093167701864
Classification Report:

	precision	recall	f1-score	support
0.0	0.77	0.91	0.83	516
1.0	0.91	0.79	0.85	1543
2.0	0.77	0.89	0.83	678
accuracy			0.84	2737
macro avg	0.82	0.86	0.84	2737
weighted avg	0.85	0.84	0.84	2737

Confusion Matrix:
[[468 48 0]
 [141 1225 177]
 [0 76 602]]



Hyperparameter Tuning

We apply GridSearchCV to tune Random Forest with a parameter grid:

- n_estimators: [100, 200]
- max_depth: [5, 10]
- min_samples_split: [2, 5]

We then re-evaluate the tuned model to check performance improvements.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10],
    'min_samples_split': [2, 5]
}

grid = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=3,
    scoring='accuracy',
    n_jobs=-1
)

grid.fit(X_train, y_train)

print("\n Best Parameters found for Random Forest:")
print(grid.best_params_)

best_rf = grid.best_estimator_
y_pred_rf = best_rf.predict(X_test)

print("\n Random Forest Performance after Tuning:")
print("Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Classification Report:\n", classification_report(y_test, y_pred_rf))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
```



Best Parameters found for Random Forest:
{'max_depth': 10, 'min_samples_split': 5, 'n_estimators': 100}

Random Forest Performance after Tuning:
Accuracy: 0.8443551333576909
Classification Report:

	precision	recall	f1-score	support
0.0	0.76	0.93	0.84	516
1.0	0.92	0.79	0.85	1543
2.0	0.78	0.90	0.83	678
accuracy			0.84	2737
macro avg	0.82	0.88	0.84	2737
weighted avg	0.86	0.84	0.84	2737

Confusion Matrix:

```
[[ 482  34   0]
 [ 150 1217 176]
 [   0   66 612]]
```

```
import pandas as pd
```

```
data = {
    'Model': ['Logistic Regression', 'Random Forest', 'XGBoost', 'Gradient Boosting', 'AdaBoost', 'LightGBM', 'CatBoost'],
    'Accuracy': [0.72, 0.86, 0.85, 0.84, 0.77, 0.85, 0.85],
    'Strengths': [
        'Fast, interpretable, good baseline',
        'Excellent accuracy, robust, strong recall',
        'High accuracy, handles imbalance',
        'Strong performance, balances precision/recall',
        'Decent on class 2, simple boosting',
        'Fast, scalable, good on class 0 and 2',
    ]
}
```

```
'Great with categorical data, balanced performance'
],
'Limitations': [
    'Weak on non-linear patterns, struggled on class 2',
    'Less interpretable, some misclassification on class 1',
    'Needs tuning, some complexity',
    'Slightly lower class 1 recall, tuning sensitive',
    'Lower accuracy, sensitive to noise',
    'Needs tuning, sensitive to data format',
    'Needs more RAM, slower on CPU'
]
}

df_results = pd.DataFrame(data)
df_results
```



	Model	Accuracy	Strengths	Limitations
0	Logistic Regression	0.72	Fast, interpretable, good baseline	Weak on non-linear patterns, struggled on class 2
1	Random Forest	0.86	Excellent accuracy, robust, strong recall	Less interpretable, some misclassification on ...
2	XGBoost	0.85	High accuracy, handles imbalance	Needs tuning, some complexity
3	Gradient Boosting	0.84	Strong performance, balances precision/recall	Slightly lower class 1 recall, tuning sensitive
4	AdaBoost	0.77	Decent on class 2, simple boosting	Lower accuracy, sensitive to noise
5	LightGBM	0.85	Fast, scalable, good on class 0 and 2	Needs tuning, sensitive to data format
6	CatBoost	0.85	Great with categorical data, balanced performance	Needs more RAM, slower on CPU



Next steps: [Generate code with df_results](#) [View recommended plots](#) [New interactive sheet](#)

```
import pandas as pd
import matplotlib.pyplot as plt

importances = pd.Series(best_rf.feature_importances_, index=X.columns)

importances.sort_values(ascending=True).plot(kind='barh', figsize=(8, 6))
plt.title('Feature Importance - Random Forest')
plt.xlabel('Importance Score')
plt.ylabel('Feature')
plt.show()
```